

Embedded Linux & System Software Interview Handbook

Chapter 5 - Linux Device Drivers (Part 1)

Topics: Device Driver Fundamentals, User Space vs Kernel Space, Driver Types, Linux Device Model

5.1 What is a Device Driver?

A Linux device driver is kernel software that controls a hardware device and provides a standard interface between applications and hardware. Applications interact with drivers using system calls rather than directly accessing hardware registers.

Data Flow

Application → libc → System Call → VFS → Device Driver → Hardware

5.2 User Space vs Kernel Space

User Space	Kernel Space
Applications	Linux Kernel
No direct hardware access	Full hardware access
Limited privileges	Privileged execution
Crash affects one process	Bug can panic the kernel

5.3 Driver Types

Character Drivers: UART, GPIO
Block Drivers: eMMC, SSD, HDD
Network Drivers: Ethernet, Wi-Fi
Platform Drivers: SoC peripherals
USB Drivers: Keyboards, Cameras
PCI Drivers: Graphics, NVMe

5.4 Linux Device Model

The Linux device model consists of buses, devices and drivers. During boot, devices are discovered and matched to registered drivers using Device Tree, ACPI, PCI IDs or USB IDs. When a match occurs, the driver's probe() function is called.

Driver Life Cycle

Register Driver → Detect Device → Match → probe() → Operational → remove() → Unload

Interview Questions

1. What is a device driver?
2. Explain user space vs kernel space.
3. Character vs Block drivers?
4. What is the Linux device model?

5. When is probe() called?
6. Why are drivers required?

Key Takeaways

Drivers isolate hardware details from applications. Understanding the Linux device model and the driver life cycle is fundamental for embedded Linux and BSP interviews.